Guardrailed Task and Motion Planning: System Overview

I. Core Architectural Principles

P1: Separation of Concerns via Layered Abstraction
Domain construction is decomposed into strictly ordered layers, each addressing a distinct abstraction level. This enables targeted diagnosis and repair when failures occur.

P2: Continuous Perception as a Service
Scene understanding runs continuously or on-demand, updating the geometric representation Ψ as new observations become available. This decouples symbolic reasoning from perception latency.

P3: Hierarchical Failure Recovery
Failures at different stages (symbolic planning, geometric feasibility, primitive execution) are diagnosed and routed to the appropriate abstraction layer for repair, avoiding unnecessary regeneration of validated components.

P4: Monotonic Domain Extension and Reuse
Domain knowledge (predicates, action schemas, grounding rules, skill specifications) persists across tasks via a Domain Knowledge Base. New tasks extend existing knowledge rather than regenerating from scratch.

P5: Active Exploration and Execution History
The system operates in partially observable environments where relevant objects, surfaces, and spatial relationships may not be visible in the initial scene. When the current scene representation Ψ is insufficient to ground a plan, the system generates exploratory actions to extend Ψ. All action attempts — successful or failed — are recorded in a persistent execution history that informs subsequent planning, decomposition, and exploration decisions.

II. System Architecture and Information Flow

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                Domain Knowledge Base (DKB)
│
│  Persists across tasks: predicates, schemas, grounding
│
│  rules, skill specifications, object-class knowledge,
│
│  exploration strategies
│
└──────────────────────────────────────────────────────────────────────────────┘
        ↕ (query / extend)
┌──────────────────────────────────────────────────────────────────────────────┐
│            Execution History Log
│
│  Records all action attempts, outcomes, and failure context
│
│  Persists within and across tasks
│
│  Fed to VLM during skill decomposition and re-decomposition
│
└──────────────────────────────────────────────────────────────────────────────┘
        ↕ (read / append)
┌──────────────────────────────────────────────────────────────────────────────┐
│            Continuous Perception Service (Ψ)
│
│  Asynchronously updates scene representation
│
│  Multiple backend options (see §IV)
```

```
│
└──────────────────────────────────────────────────────────────────────────────
──┘
        ↓ (provides scene data)
┌──────────────────────────────────────────────────────────────────────────────
──┐
│           Layered Domain Generation
│
│  L1: Goal → L2: Predicates → L3: Action Schemas (pure PDDL)
│
│  → L4: Grounding + Skill Specifications → L5: Initial State
│
│  Each layer validated before proceeding
│
│  DKB queried first; only missing components generated
│
│  Missing entities trigger exploration before L5
│
└──────────────────────────────────────────────────────────────────────────────
──┘
        ↓
┌──────────────────────────────────────────────────────────────────────────────
──┐
│           Classical / Contingent Planner
│
│  Produces symbolic action sequence
│
│  Branches on sensing outcomes for uncertain predicates
│
└──────────────────────────────────────────────────────────────────────────────
──┘
        ↓
┌──────────────────────────────────────────────────────────────────────────────
──┐
│           Geometric Feasibility Verification
│
```

```
│  Checks motion planning constraints using grounded
│
│  skill specifications from L4
│
└──────────────────────────────────────────────────────────────────────────────
──┘
        ↓
┌──────────────────────────────────────────────────────────────────────────────
──┐
│           Primitive Execution with Monitoring
│
│  Executes motions, monitors for failures
│
│  Logs all outcomes to Execution History
│
└──────────────────────────────────────────────────────────────────────────────
──┘
        ↓
    ┌───┴───┐
    ↓       ↓
 SUCCESS  FAILURE → Failure Diagnosis → Recovery Routing
                    → Skill re-decomposition (L4-level)
                    → Exploration if entities missing
                    → Routes to appropriate repair layer
                    → Updates DKB with learned repairs
```

III. Layered Domain Generation

3.1 Layer Overview

Each layer undergoes validation before its artifact is locked. All validation checks are algorithmically implementable — each criterion below includes its implementation strategy. Failed validation triggers targeted re-generation of the current layer with specific feedback, not escalation to upstream layers.

Layer	Input	Output	Reusable?
L1: Goal Specification	Natural language task τ_nl	Goal formula G = {g_1, g_2, ...}	Per-task (not reusable)
L2: Predicate Vocabulary	Goal G, task τ_nl, DKB	Minimal predicate set P	**Yes** — accumulative library
L3: Action Schemas	Goal G, Predicates P, task τ_nl, DKB	Pure PDDL action definitions (no execution detail)	**Yes** — action library
L4: Grounding + Skill Specifications	Predicates P, Actions A, Scene Ψ, DKB	Symbol-to-scene mappings + skill specifications (PtP JSON artifacts)	**Partially** — grounding rules reusable per environment; skill specifications reusable via re-grounding
L5: Initial State Construction	Grounded symbols from L4, Scene Ψ	Initial PDDL state P_init	Per-task snapshot (not reusable)

Layer Dependencies (DAG):

```
L₁ → L₂ → L₃ → L₄ → L₅
                 ↑
                 Ψ (perception service)

L₄ depends on: {L₂, L₃, Ψ}
L₅ depends on: {L₂, L₄}
```

3.2 Layer L_1: Goal Specification

Input: Natural language task description τ_nl
Output: Set of goal predicates G = {g_1, g_2, ...}
Dependencies: None
Reuse: Per-task (not reusable)

The LLM extracts abstract goal conditions from the natural language task description without defining predicates or actions. Goal predicates describe desired end-state conditions expressed as predicate-name + argument structures. At this stage, the predicates referenced in G are provisional — they will

be formally defined in L_2. L_1 establishes *what* the system should achieve; all subsequent layers determine *how*.

3.2.1 L_1 Validation

Input to validator: Goal formula $G = \{g_1, g_2, ...\}$ produced by LLM from τ_nl

Validation checks:

- **Non-emptiness:**
  - *Algorithm:* `assert len(G) > 0`. Trivial length check on the goal set.
- **Well-formedness:**
  - *Algorithm:* Parse each g_i against a PDDL goal grammar: `predicate-name(arg₁, arg₂, ...)`. Regex or parser validates structure: predicate name must be a non-empty lowercase-hyphenated string, arguments must be non-empty strings, parentheses must balance. Specifically: `match(gᵢ, r"^\([a-z][a-z0-9-]* [a-z][a-z0-9_-]*( [a-z][a-z0-9_-]*)*\)$")`. Reject any g_i that fails the parse.
- **Internal consistency:**
  - *Algorithm:* Maintain a registry of *uniqueness-constrained predicates* — predicates where a specific argument can only take one value at a time (e.g., `object-at` constrains an object to one location). This registry is pre-defined for the domain class (tabletop manipulation, navigation, etc.). For each uniqueness-constrained predicate in G, group goal predicates by the constrained argument and check that no argument appears with two different values. Implementation: `for pred in uniqueness_constrained: groups = group_by(G, pred, constrained_arg_index); for group in groups: assert len(set(group.values)) <= 1`. The uniqueness constraint registry is a static configuration file listing predicates and which argument positions are uniquely constrained (e.g., `{predicate: "object-at", unique_arg: 0, varying_arg: 1}` means argument 0 can only map to one value of argument 1).
- **Argument plausibility:**
  - *Algorithm:* Extract all argument strings from G. Compare against a union of: (a) object labels present in Ψ.objects (from perception), (b) surface/region labels present in Ψ.surfaces, (c) the robot identifier. Any argument string not present in any of these sets is flagged.

Implementation: `known_entities = set(Ψ.objects.keys()) ∪ set(Ψ.surfaces.keys()) ∪ {robot_id}; for g in G: for arg in g.arguments: if arg not in known_entities: flag(g, arg, "unknown entity")`. Flagged goals are returned to the LLM with the list of known entities for re-generation. This is a concrete entity-existence check against the current scene rather than a semantic judgment.

Auto-repair: None. Goal failures return the specific failed check and message to the LLM for re-generation.

On success: G is locked as the L_1 artifact. All downstream layers must express goals using exactly the predicates that will be defined in L_2 to cover G.

3.3 Layer L_2: Predicate Vocabulary

Input: Goal G from L_1, task τ_nl, DKB predicate library
Output: Minimal predicate set $P = \{p_1, p_2, ...\}$
Dependencies: L_1
Reuse: Accumulative — predicates persist in DKB across tasks

The LLM generates a minimal predicate vocabulary sufficient to:

1. Express all goal conditions in G
2. Express initial state from scene perception Ψ
3. Express state transitions needed to achieve G (preconditions and effects of actions that will be defined in L_3)

Each predicate is annotated with a type classification:

- **State predicates:** Directly controlled by robot actions (e.g., `robot-at`, `gripper-empty`, `object-at`). Truth values are deterministically known.
- **Sensed/external predicates:** Properties that depend on the external world and cannot be assumed without verification (e.g., `object-graspable`, `path-clear`, `door-locked`). These require corresponding checked variants.
- **Checked predicates:** State predicates of the form `checked-<base-predicate>` that track whether a sensing action has verified the corresponding sensed predicate. Always initialized to FALSE.

When the DKB predicate library already contains predicates relevant to the current task, the system queries existing predicates first and generates only those not

already present in the library.

3.3.1 L_2 Validation

Input to validator: Predicate set P = {p_1, p_2, ...} produced by LLM given G (from L_1) and τ_nl. Each predicate includes: name, argument types, and type classification (state / sensed / checked).

Validation checks:

- **Goal coverage:**
  - *Algorithm:* Extract predicate names from G (string match on the predicate-name portion of each g_i). Check set membership: `goal_pred_names = {predicate_name(g) for g in G}; P_names = {p.name for p in P}; missing = goal_pred_names - P_names; assert len(missing) == 0`. If missing is non-empty, return the specific missing predicate names to the LLM.
- **Naming compliance:**
  - *Algorithm:* Regex match each predicate name against the PDDL naming pattern: `match(p.name, r"^[a-z][a-z0-9]*(-[a-z0-9]+)*$")`. Reject any predicate failing the pattern. Additionally verify argument type names follow the same pattern.
- **Checked-variant completeness (Invariant I1):**
  - *Algorithm:* Filter P for predicates with type classification `sensed` or `external`. For each such predicate p, check that a predicate named `checked-{p.name}` exists in P with type classification `state` and matching argument types. Implementation: `sensed_preds = {p for p in P if p.type in ("sensed", "external")}; for p in sensed_preds: expected_name = f"checked-{p.name}"; matches = [q for q in P if q.name == expected_name and q.arg_types == p.arg_types]; assert len(matches) == 1`. Missing checked variants trigger auto-repair (see below).
- **Type consistency:**
  - *Algorithm:* Build a type assignment map: for each argument position across all predicates, record which type is assigned. Then check that no entity name is assigned conflicting types across different predicates. Implementation: `type_map = {}; for p in P: for arg_name, arg_type in p.arguments: if arg_name in type_map: assert type_map[arg_name] == arg_type; else: type_map[arg_name] = arg_type`. Also verify all argument types belong to a pre-

defined set of valid PDDL types for the domain (e.g., `{"object", "location", "surface", "robot", "region"}`).

- **Minimality:**
  - *Algorithm:* This cannot be fully validated before L_3 (actions don't exist yet). Instead, perform a deferred check: after L_3 validation, revisit P and compute used predicates as the union of predicates appearing in G, any φ_pre, any φ_eff, and any initial state construction rule. `used = goal_preds ∪ precondition_preds ∪ effect_preds; unused = P_names - used`. Unused predicates are removed from P automatically (they cannot affect planning). This converts the vague "minimality" check into a concrete dead-predicate elimination pass run after L_3 completes. The deferred nature is safe because removing unused predicates cannot invalidate L_3 — if a predicate appears nowhere in actions or goals, its absence has no effect on planning.

Auto-repair: Missing checked variants are automatically generated: for each sensed predicate p missing its checked variant, instantiate `checked-{p.name}` with identical argument types, type classification `state`, and initial value FALSE. No LLM call required.

On success: P is locked as the L_2 artifact. New predicates are committed to the DKB predicate library. L_3 must use only predicates from P.

3.4 Layer L_3: Action Schema Generation

Input: Goal G (L_1), Predicates P (L_2), task τ_nl, DKB action schema library
Output: Action schema set A = {a_1, a_2, ...} — pure PDDL action definitions with no execution detail
Dependencies: L_1, L_2
Reuse: Action schemas persist in DKB across tasks

L_3 outputs abstract PDDL action definitions. All execution strategy — primitive sequences, semantic parameters, geometric constraints — is deferred to L_4 where a VLM reasons about the specific scene and object to determine how each action is physically realized. This separation ensures the symbolic planning layer is independent of scene-specific execution concerns, and that L_4 retains full flexibility to adapt execution to novel objects and mechanisms.

Action Schema Structure:

```
ActionSchema(α) = (name, parameters, φ_pre, φ_eff)

where:
  name:       Unique action identifier
  parameters: Typed variables [(v₁: T₁), (v₂: T₂), ...]
  φ_pre:      Precondition formula over predicates from L₂
  φ_eff:      Effect formula over predicates from L₂
```

Action schemas are purely symbolic. They define *what state transition* an action accomplishes (via preconditions and effects) without specifying *how* that transition is physically executed. The same schema `open(?obj)` applies whether the object is a twist-cap bottle, a pull-cap marker, or a hinged cabinet — the physical execution strategy is determined at L_4 by the VLM reasoning over the scene.

The LLM generates actions using only predicates from the validated L_2 artifact. When the DKB action schema library already contains schemas relevant to the current task (matched by name and predicate compatibility), those schemas are reused and only missing action types are generated.

3.4.1 L_3 Validation

Input to validator: Action schema set $A = \{a_1, a_2, ...\}$ produced by LLM given G (L_1), P (L_2), and τ_nl. Each schema contains (name, parameters, φ_pre, φ_eff).

Validation checks:

- **Symbolic closure:**
  - *Algorithm:* Parse all predicate references from φ_pre and φ_eff of every action (PDDL formula parser extracts predicate names). Check set membership against P. `for a in A: preds_used = extract_predicates(a.φ_pre) ∪ extract_predicates(a.φ_eff); violations = preds_used - P_names; assert len(violations) == 0`. If violations exist, return the specific undefined predicates and the actions that reference them.
- **Parameter consistency:**

- *Algorithm:* For each action schema, extract all variable references from φ_pre and φ_eff (PDDL variables match `?[a-z][a-z0-9_-]*`). Check that every referenced variable is declared in the action's parameter list. `for a in A: declared = {v.name for v in a.parameters}; referenced = extract_variables(a.φ_pre) ∪ extract_variables(a.φ_eff); undeclared = referenced - declared; assert len(undeclared) == 0`. Additionally verify each declared parameter has a type from the type set established in L_2.

- **Sensing coverage (Invariant I1 enforcement):**
  - *Algorithm:* Collect all `checked-*` predicates appearing in any precondition across A. For each such predicate, verify an action exists in A whose effects include setting that predicate to TRUE. `checked_in_pre = {p for a in A for p in extract_predicates(a.φ_pre) if p.startswith("checked-")}; produced_by_effects = {p for a in A for p in extract_positive_effects(a.φ_eff) if p.startswith("checked-")}; uncovered = checked_in_pre - produced_by_effects; assert len(uncovered) == 0`. Uncovered checked predicates trigger auto-repair (sensing action generation).

- **Goal achievability (reachability analysis):**
  - *Algorithm:* Construct a *relaxed planning graph* (delete-relaxation). This is a standard polynomial-time algorithm used in classical planning heuristics:
    1. Initialize fact layer F_0 with all predicates that could be true in any initial state (all state predicates that can be set by grounding rules + all checked predicates set to both TRUE and FALSE in the relaxed version).
    2. Expand: For each action whose relaxed preconditions are satisfied by current fact layer, add all positive effects to the next fact layer. `Fᵢ₊₁ = Fᵢ ∪ {e⁺ for a in A if relaxed_pre(a) ⊆ Fᵢ for e⁺ in positive_effects(a)}`.
    3. Repeat until fixpoint (no new facts added) or all goal predicates in G are present.
    4. If fixpoint reached without covering G, report which goal predicates are unreachable and which actions' preconditions are never satisfiable.
  - Complexity: $O(|A| \times |P| \times |F|)$ per layer, with at most $|P| \times |\text{objects}|$ layers. Polynomial in domain size, runs in milliseconds for typical manipulation domains.

Auto-repair: Missing sensing actions are automatically generated: for each uncovered checked predicate `checked-p`, instantiate a sensing action `check-p` with precondition `(robot-near ?obj)` (or the appropriate proximity predicate from the predicate set) and effect `(checked-p ?obj)`. Parameters and types are inferred from the base predicate's argument structure. No LLM call required.

On success: A is locked as the L_3 artifact. New schemas are committed to the DKB action schema library. Deferred minimality check from L_2 is now executed (see §3.3.1). L_4 receives pure PDDL schemas for primitive decomposition.

3.5 Layer L_4: Symbolic Grounding and Skill Specification Generation

Input: Predicates P (L_2), Action schemas A (L_3), Scene representation Ψ, DKB (grounding rules + skill specification library)
Output: Two coupled artifacts — symbol-to-scene grounding mappings and skill specifications (primitive decompositions) for each action
Dependencies: L_2, L_3, Ψ (perception service)
Reuse: Grounding rules reusable per environment; skill specifications cached in DKB per (action_name, object_instance/class) and reusable via re-grounding

L_4 handles both symbol grounding and skill specification generation. The skill specification for each action is produced by a VLM that reasons over the PDDL action schema, the annotated scene representation (surface maps and interaction maps from Ψ), and the primitive library to generate a primitive sequence with semantic and situational parameter bindings. This follows the Prompt-to-Primitives approach: the VLM is not selecting from pre-defined templates but rather composing a novel primitive sequence appropriate for the specific object, mechanism, and scene context.

This design preserves full flexibility — the VLM can generate different primitive sequences for the same abstract action (e.g., `open`) depending on the object's mechanism, geometry, and scene configuration. A stuck bottle cap might require multiple twist-release-retighten-twist cycles; a cabinet with a magnetic latch might need an initial press before a pull. These are not pre-enumerated templates but emerge from the VLM's reasoning about the specific scene.

3.5.1 Artifact 4a: Grounding Mappings

For each predicate p ∈ P and action α ∈ A, establish symbol-to-scene bindings using pre-defined grounding rules:

```
ground_predicate(p) → grounding_rule
ground_action_parameters(α) → binding_rules

Pre-defined grounding rules map PDDL types to Ψ element type
s:
  object → Ψ.objects[id]
  location/surface → Ψ.surfaces[id]
  robot → Ψ.robot
  region → Ψ.regions[id]

These rules are static per domain class and do not require
LLM generation. The system applies them deterministically.
```

Grounding rules are pre-defined mappings from PDDL types to scene element categories. They are part of the system configuration, not LLM-generated artifacts. The rules define *how to interpret* Ψ for a given parameter type — concrete bindings are applied at planning/execution time using the latest Ψ state.

3.5.2 Artifact 4b: Skill Specifications via VLM Primitive Decomposition

For each PDDL action schema α in the plan, the VLM generates a **skill specification** Φ — the PtP manipulation specification — by reasoning over the action's intent (from the schema name and effects), the annotated scene representation (surface maps + interaction maps from Ψ), and the primitive library.

Primitive Library (P):

The executable vocabulary consists of six atomic manipulation primitives (AMPs). Each primitive defines a structured parameter schema specifying required and optional parameters with their valid types/values:

```
Primitive                | Parameters
-------------------------|-------------------------------------
```

```
--------
move_gripper_to_pose    | target: interaction_point label (re
quired)
                        | grasp_mode: {top_down, side} (requi
red)
                        | approach_offset: float (meters, opt
ional)
close_gripper           | (none)
open_gripper            | (none)
push_pull               | surface: surface label (required)
                        | force_direction: {perpendicular, pa
rallel}
                        | interaction_type: {press, sustaine
d}
                        | articulation_mode: {linear, revolut
e}
                        | hinge_boundary: surface boundary la
bel
                        |   (required if articulation_mode=re
volute)
twist                   | direction: {cw, ccw} (required)
                        | angle: float (radians, optional)
retract_gripper         | (none — returns to home configurati
on)
turn_base               | direction: {left, right} (required)
                        | angle: float (radians, optional)
scan_region             | n_observations: int (default: 4)
                        | (rotates in place, capturing n obse
rvations
                        |  at evenly spaced angles, triggers
Ψ update
                        |  after each observation)
observe                 | target_region: region/surface label
(required)
                        | (triggers targeted Ψ update on the
specified
```

```
                          |  region without physical contact)
move_base_to_pose         | target: location label (required)
                          | (mobile base navigation — available
only
                          |  when mobile base is present)
```

The first six primitives (move_gripper_to_pose through retract_gripper) form the manipulation vocabulary from the Prompt-to-Primitives framework. The remaining four (turn_base, scan_region, observe, move_base_to_pose) extend the library with exploration and observation capabilities that enable the system to actively expand its scene representation Ψ when operating in partially observable environments.

Execution History Log:

All action attempts are recorded in a persistent execution history that accumulates within and across tasks:

```
ExecutionHistoryLog = {
  entries: [
    {
      action_name: string,         # PDDL action that was att
empted
      target_object: string,       # Object the action target
ed
      skill_specification: Φ,      # The (π, θ_sem, θ_sit) th
at was executed
      outcome: {success, failure},
      failure_context: {           # Populated only on failur
e
        failure_type: {F1, F2, F3, F4, F5},
        error_detail: string,
        scene_state_at_failure: Ψ_snapshot
      },
      Ψ_before: Ψ_snapshot,       # Scene state before execut
ion
      Ψ_after: Ψ_snapshot,        # Scene state after executi
```

```
on
      timestamp: datetime
    },
    ...
  ]
}
```

The execution history serves three purposes:
1. **Preventing repeated failures:** The VLM sees what was previously attempted and what failed, avoiding re-generating the same failing strategy.
2. **Informing exploration:** For search-type actions, the history records which locations have been checked and what was found (or not found), enabling systematic search rather than random exploration.
3. **Improving future decomposition:** Successful and failed strategies are associated with object classes and action types, feeding into the DKB's object class knowledge for future tasks.

VLM Decomposition Process:

The VLM is prompted with:
1. The PDDL action schema (name, parameters, effects — so it understands the *intent*)
2. The annotated surface map and interaction map from Ψ (so it can reference specific scene elements by label)
3. The primitive library schema (so it knows what primitives are available and what parameters each requires)
4. The target object's properties from Ψ (geometry, detected interaction points, surfaces, articulation information if available)
5. **Relevant execution history entries** (previous attempts at this action or similar actions on this object/class, including outcomes and failure contexts)

The VLM produces a skill specification:

```
SkillSpecification(Φ) = (π, θ_sem, θ_sit)

where:
  π:      Ordered primitive sequence [p₁, p₂, ..., pₙ]
          Composed by the VLM — any valid ordering of primiti
```

```
ves
          from the library, of any length

  θ_sem:  Semantic parameters per primitive — discrete choice
s
          selected from enumerated valid values in the librar
y
          schema for each primitive
          Example: {grasp_mode: "side", force_direction:
                    "perpendicular"}

  θ_sit:  Symbolic situational references — the VLM selects
          labels from the annotated scene maps, which are the
n
          resolved to concrete geometric values via grounding
          rules
          Example for open(bottle):
            move_gripper_to_pose.target = "bottle_cap"
            → grounding: Ψ.objects["bottle_1"]
                .interaction_points["bottle_cap"].pose
            push_pull.surface = "surface_A"
            → grounding: Ψ.surfaces["surface_A"].plane_eq
```

What the VLM controls vs. what is automated:

Responsibility	Owner
Primitive sequence composition (which primitives, what order, how many)	VLM
Semantic parameter selection (grasp mode, force direction, twist direction, etc.)	VLM
Symbolic label selection for situational parameters (which interaction point, which surface)	VLM
Resolving symbolic labels to concrete poses/geometry	Pre-defined grounding rules (automated)
Instantiating geometric constraints from primitive schemas	Primitive library (automated)

Responsibility	Owner
Evaluating geometric feasibility	Motion planner (automated)

This division ensures the VLM operates at the semantic reasoning level — choosing *what* to do and *where* — while all geometric computation is handled by deterministic subsystems. The VLM's output is fully constrained to the vocabularies exposed by the primitive library (valid primitives, valid parameter values) and the scene representation (valid labels), making it algorithmically validatable despite being freely composed.

DKB Skill Specification Caching:

After successful validation and execution, the skill specification Φ is cached in the DKB:

```
DKB.skill_specifications[(action_name, object_id)] = {
  specification: Φ,
  object_class: Ψ.objects[target].class,
  source_scene_hash: hash(Ψ),
  success: bool,
  timestamp: datetime
}
```

For subsequent tasks involving the same action on the same or similar objects:
1. **Same object instance:** Reuse full Φ, re-execute grounding resolution only (update Θ_sit with fresh Ψ poses). No VLM call.
2. **Same object class, different instance:** Reuse π and Θ_sem from cached Φ, re-select Θ_sit labels for the new instance's interaction points/surfaces, re-ground. May require VLM call only if the new instance's interaction point labels differ from the cached specification.
3. **Novel object class:** Full VLM decomposition required.

This is the PtP re-grounding advantage: the structural specification (π, Θ_sem) remains valid across scene changes; only the continuous parameter bindings need updating.

Skill Re-Decomposition Loop:

When a skill specification fails during execution — the primitive sequence ran but did not achieve the expected PDDL effects — the system does not immediately escalate to symbolic replanning or domain repair. Instead, it first attempts **skill-level re-decomposition**: generating a new skill specification for the same PDDL action, informed by the failure context.

```
re_decompose_skill(α, Ψ_current, history, max_attempts=3):

  for attempt in range(max_attempts):
    # Gather relevant history for this action
    relevant_history = history.get_entries(
      action=α.name,
      target=α.target_object,
      include_class_matches=True
    )

    # VLM generates new specification with failure context
    Φ_new = vlm_decompose(
      action_schema = α,
      scene = Ψ_current,
      primitive_library = AMP_LIBRARY,
      execution_history = relevant_history,  # What was trie
d, what failed
      feedback = "Previous specification failed: {last_failur
e_detail}"
    )

    # Validate new specification
    if not validate_skill_spec(Φ_new):
      history.append(attempt_record(Φ_new, "validation_faile
d"))
      continue

    # Execute
    result = execute_skill(Φ_new)
    history.append(attempt_record(Φ_new, result))
```

```
    # Update Ψ after execution (regardless of outcome)
    Ψ_current = perception_service.get_current()

    if result.success:
      DKB.cache_skill(α, Φ_new, success=True)
      return SUCCESS

  # All attempts exhausted — escalate
  return ESCALATE_TO_RECOVERY
```

This loop is the first line of defense before engaging the heavier recovery machinery. It handles cases where the abstract action is correct but the physical execution strategy was wrong — a common scenario with novel objects where the VLM's first attempt may not account for an unusual mechanism or geometry.

Exploration as Entity Resolution:

When L_4 validation fails because a goal-referenced entity is missing from Ψ (the entity coverage check), the system enters an exploration phase before declaring failure. Exploration is modeled as a PDDL action that goes through the normal $L_3 \rightarrow L_4$ pipeline:

At L_3, the LLM generates exploration actions as needed:

```
(:action find
  :parameters (?obj - object)
  :precondition (and (not (object-located ?obj)))
  :effect (and (object-located ?obj)
               (checked-object-present ?obj)))
```

At L_4, the VLM decomposes `find(?obj)` into an exploratory skill specification using the exploration primitives and reasoning about likely object locations based on the current scene:

```
Example: find(milk) with fridge visible in Ψ

VLM reasoning: "Milk is commonly stored in refrigerators.
```

```
  A fridge is visible in the scene at surface_B."

Φ = {
  π: [move_base_to_pose, open_gripper, move_gripper_to_pose,
      close_gripper, push_pull, open_gripper, observe,
      retract_gripper],
  θ_sem: {push_pull.force_direction: "perpendicular",
          push_pull.articulation_mode: "revolute", ...},
  θ_sit: {move_base_to_pose.target: "fridge_front",
          move_gripper_to_pose.target: "fridge_handle",
          push_pull.hinge_boundary: "fridge_hinge",
          observe.target_region: "fridge_interior"}
}
```

After execution, the perception service updates Ψ. If `milk` now appears in `Ψ.objects`, the `find` action's effects are satisfied and planning continues. If not, the skill re-decomposition loop tries alternative strategies:

```
Attempt 1 failed: milk not in fridge
  → history records: {find(milk), checked fridge, not found}

Attempt 2: VLM sees history, proposes checking counter
  → π: [scan_region, observe]
  → θ_sit: {observe.target_region: "kitchen_counter"}

Attempt 2 failed: milk not on counter
  → history records: {find(milk), checked fridge + counter, n
ot found}

Attempt 3: VLM proposes checking pantry/cabinet
  → π: [move_base_to_pose, move_gripper_to_pose, close_grippe
r,
         push_pull, observe, retract_gripper]
  → decomposes into open(cabinet) + observe(interior)
```

If all re-decomposition attempts fail, the system escalates: the `find` action is marked as failed in the symbolic plan, and the planner is invoked with the updated belief state. This may result in the goal being declared unreachable (if the object is essential and cannot be found) or an alternative plan that doesn't require the missing object.

Exploration budget: Re-decomposition attempts for exploration actions count as 0.5 iterations in the bounded recovery loop (same as replanning). This allows multiple exploration attempts within the iteration budget while still guaranteeing termination.

3.5.3 L_4 Validation

L_4 produces two coupled artifacts and validates both. The VLM's output is constrained to the primitive library's vocabulary and the scene's label set, making validation algorithmic despite the free-form composition.

Artifact 4a Validation — Grounding Mappings:

Since grounding rules are pre-defined (not LLM-generated), validation of the rules themselves is a one-time system configuration check. The runtime validation checks that grounding can be *applied* to the current scene:

- **Grounding completeness:**
  - *Algorithm:* For each predicate parameter and action parameter in the domain, look up the pre-defined grounding rule by PDDL type. Verify the rule exists: `for p in P: for arg in p.arguments: assert arg.type in GROUNDING_RULES`. Since grounding rules are pre-defined per domain class, this check will only fail if the LLM introduced a novel PDDL type not covered by the configuration — which is caught by the L_2 type consistency check. This validation is therefore a redundant safety net and runs as a simple dict lookup per argument.
- **Scene element availability:**
  - *Algorithm:* For each grounding rule that maps to a Ψ element category, verify that Ψ contains at least one element of that category. `for rule in GROUNDING_RULES.values(): element_type = rule.target_category; assert len(Ψ.get_elements(element_type)) > 0`. For example, if actions reference `location` typed parameters that ground to `Ψ.surfaces`, verify Ψ.surfaces is

non-empty. This catches perception failures where expected scene elements were not detected.

- **Entity coverage:**
  - *Algorithm:* For each object constant referenced in the goal G and initial state construction, verify the entity exists in Ψ. `goal_entities = extract_entity_names(G); for entity in goal_entities: assert entity in Ψ.objects or entity in Ψ.surfaces or entity == robot_id`. Missing entities indicate either a perception failure or a goal that references nonexistent objects.

Artifact 4b Validation — Skill Specifications:

- **Primitive membership:**
  - *Algorithm:* Every primitive in the VLM-generated sequence must exist in the primitive library. `for p in τ.π: assert p in AMP_LIBRARY.primitives`. This is a direct set membership check. The primitive library defines the complete executable vocabulary — the VLM cannot invent primitives that don't exist.
- **Semantic parameter validity:**
  - *Algorithm:* For each primitive in the sequence, the library defines the parameter schema including enumerated valid values for each semantic parameter. Check that the VLM's selections are within bounds: `for i, p in enumerate(τ.π): schema = AMP_LIBRARY.primitives[p].param_schema; for param_name, selected_value in τ.θ_sem[i].items(): assert param_name in schema; assert selected_value in schema[param_name].valid_values`. This is a direct enumeration membership check per parameter. The VLM can freely compose any sequence it wants, but each primitive's parameters must conform to that primitive's schema.
- **Symbolic reference resolution:**
  - *Algorithm:* For each situational parameter where the VLM selected a symbolic label (e.g., "bottle_cap", "handle", "surface_A"), verify the label resolves to a concrete element in Ψ. `for i, p in enumerate(τ.π): schema = AMP_LIBRARY.primitives[p].param_schema; for param_name, symbolic_label in τ.θ_sit[i].items(): param_type = schema[param_name].grounding_type; if param_type == "interaction_point": target_obj = get_bound_object(α, τ); assert symbolic_label in Ψ.objects[target_obj].interaction_points; elif param_type == "surface": assert symbolic_label in Ψ.surfaces; elif param_type == "affordance_region": assert symbolic_label in Ψ.objects[target_obj].affordance_regions`. This checks that the

perception system has detected and labeled the specific part/point the VLM is referencing. If the label doesn't exist in Ψ, the failure is routed to either: (a) a perception update request if the object exists but the part wasn't detected, or (b) an L_4 re-generation if the VLM selected a nonsensical label.

- **Required parameter completeness:**
  - *Algorithm:* For each primitive, the library schema specifies which parameters are required vs. optional. Check that all required parameters have been assigned a value (either semantic or situational): `for i, p in enumerate(τ.π): schema = AMP_LIBRARY.primitives[p].param_schema; required = {name for name, spec in schema.items() if spec.required}; provided = set(τ.θ_sem[i].keys()) ∪ set(τ.θ_sit[i].keys()); missing = required - provided; assert len(missing) == 0`.
- **Constraint instantiation:**
  - *Algorithm:* Each primitive in the library defines constraint templates (e.g., `reachable(target)`, `collision_free(path_to(target))`). After symbolic references are resolved to poses via grounding rules, instantiate all constraint templates with concrete values and verify they are evaluatable. `for i, p in enumerate(τ.π): constraint_templates = AMP_LIBRARY.primitives[p].constraints; for ct in constraint_templates: grounded_constraint = ct.instantiate(τ.θ_sit_grounded[i]); assert grounded_constraint.is_evaluatable()`. A constraint is evaluatable if all its pose/geometry arguments are concrete (not None, not symbolic). This does NOT evaluate whether the constraint is satisfied — that happens during geometric feasibility verification. This check only confirms the constraints *can be* evaluated.
- **Scene feasibility pre-check:**
  - *Algorithm:* For each grounded interaction point or target pose, perform lightweight workspace checks:
    1. Workspace containment: `assert robot.workspace_bounds.contains(target_pose.position)`. Uses the robot's pre-computed workspace bounding volume (sphere or convex hull). O(1) check.
    2. Gross reachability: `assert np.linalg.norm(target_pose.position - robot.base_position) <= robot.max_reach`. Simple distance check against max arm reach.

3. Bounding box grasp feasibility (for primitives involving `close_gripper` preceded by `move_gripper_to_pose`): `obj_bbox = Ψ.objects[target].geometry.bbox; gripper_max_aperture = robot.gripper.max_aperture; if θ_sem.grasp_mode == "top": assert obj_bbox.min_dimension("xy") <= gripper_max_aperture; elif θ_sem.grasp_mode == "side": assert obj_bbox.min_dimension("z_cross_section") <= gripper_max_aperture`. Checks whether the object's dimensions are physically compatible with the selected grasp mode, using bounding box approximation. O(1) per object.

- These are necessary-but-not-sufficient conditions. Passing them does not guarantee geometric feasibility, but failing them guarantees infeasibility — avoiding expensive motion planning calls on impossible configurations.

Auto-repair: If symbolic reference resolution fails because a label doesn't exist in Ψ: (a) trigger a targeted perception update (T2) for the relevant object and retry; (b) if the label still doesn't resolve, return the available labels for that object to the VLM for re-composition. If semantic parameter validation fails, return the valid value set to the VLM. If scene feasibility pre-check fails, report the specific geometric constraint (e.g., "object exceeds gripper aperture for top grasp") and provide this as feedback for VLM re-composition.

On success: Grounding mappings and skill specifications are locked as the L_4 artifact. Skill specifications are cached in the DKB indexed by (action_name, object_id) with object class annotation for cross-instance reuse. L_5 uses grounding rules to construct initial state.

3.6 Layer L_5: Initial State Construction

Input: Grounded symbols from L_4, Scene representation Ψ
Output: Initial PDDL state P_init
Dependencies: L_2, L_4
Reuse: Per-task snapshot (not reusable)

L_5 constructs the initial PDDL state by grounding all predicates from L_2 to truth values using the pre-defined grounding rules and the current scene representation Ψ. This is always generated fresh for each task, since the state is a snapshot of the world at task start time.

Grounding strategy by predicate type:

- **Robot state predicates** (e.g., `robot-at`, `gripper-empty`): Ground to robot's current known state. These are always deterministically known from robot state feedback.
- **Object state predicates** (e.g., `object-at`, `on`): Ground to spatial relationships observed in Ψ. These are derived from the perception service's spatial relation graph.
- **Checked predicates** (e.g., `checked-object-graspable`): Initialize to FALSE. Nothing has been verified by sensing at task start. This is invariant — checked predicates are *never* initialized to TRUE regardless of prior knowledge, because the sensing guarantee requires runtime verification.
- **Sensed/external predicates** (e.g., `object-graspable`, `door-locked`): These represent uncertain properties. In classical PDDL (without partial observability extensions), they must be initialized to a default value. The system uses conservative defaults: properties required as preconditions are initialized to FALSE (forcing the planner to include sensing actions), ensuring the sensing coverage invariant is maintained at execution time.

L_5 is largely algorithmic — given the grounding rules from L_4 and the current Ψ, truth value assignment follows deterministic rules per predicate type. The LLM is not involved in L_5; this layer is fully automated.

3.6.1 L_5 Validation

Input to validator: Initial PDDL state P_init produced by applying grounding rules to Ψ.

Validation checks:

- **State completeness:**
  - *Algorithm:* Enumerate all grounded predicate instances by computing the cross product of predicates and applicable typed objects from Ψ. For each predicate p with argument types (T_1, T_2, ...), enumerate all tuples of entities matching those types. Check that P_init assigns a truth value to each. `expected_instances = set(); for p in P: entity_tuples = cartesian_product(*[Ψ.get_entities_of_type(t) for t in p.arg_types]); for tup in entity_tuples: expected_instances.add((p.name, tup)); assigned = {(fact.predicate, fact.args) for fact in P_init}; missing = expected_instances - assigned; assert len(missing) == 0`.

For predicates using closed-world assumption (CWA), missing instances default to FALSE. If not using CWA, missing instances are flagged.

- **Grounding strategy compliance:**
  - *Algorithm:* Partition P_init facts by predicate type classification (from L_2). For each partition, verify initialization rule:
    - Checked predicates: `for fact in P_init: if fact.predicate.type == "checked": assert fact.value == FALSE`.
    - Sensed/external predicates: `for fact in P_init: if fact.predicate.type == "sensed": assert fact.value == FALSE` (conservative default).
    - Robot state predicates: Verify against robot state API: `for fact in P_init: if fact.predicate.type == "robot_state": assert fact.value == robot.query_state(fact.predicate, fact.args)`.
    - Object state predicates: Verify against Ψ spatial relations: `for fact in P_init: if fact.predicate.type == "object_state": assert fact.value == Ψ.evaluate_relation(fact.predicate, fact.args)`.
  - Each of these is a deterministic check against a known source of truth.
- **Type match verification:**
  - *Algorithm:* For each fact in P_init, verify that the entity constants in the argument positions match the declared types from the predicate definition. `for fact in P_init: pred_def = P[fact.predicate]; for i, arg in enumerate(fact.args): expected_type = pred_def.arg_types[i]; actual_type = Ψ.get_entity_type(arg); assert actual_type == expected_type or actual_type is subtype of expected_type`. Uses the entity type information stored in Ψ (each object/surface has a type annotation from the perception system).
- **Consistency with Ψ spatial relations:**
  - *Algorithm:* For all spatial predicates (object-at, on, in, etc.) in P_init, cross-reference with Ψ.spatial_relations. `spatial_preds = {p for p in P if p.is_spatial}; for fact in P_init: if fact.predicate in spatial_preds and fact.value == TRUE: assert Ψ.spatial_relations.contains(fact.predicate, fact.args)`. Mismatches indicate a bug in the grounding procedure (since L_5 is automated, mismatches are system errors rather than LLM errors).
- **Goal-state distinctness:**

- *Algorithm:* Check whether the initial state already satisfies all goal predicates. `goal_satisfied = all(P_init.evaluate(g) == TRUE for g in G); if goal_satisfied: warn("Initial state already satisfies goal — verify task is non-trivial")`. This is a warning, not a hard failure — some tasks may legitimately have no-op solutions (e.g., "verify the mug is on the table" when it already is). The warning triggers a confirmation check rather than rejection.

Auto-repair: Since L_5 is fully automated, validation failures indicate system bugs rather than LLM errors. Checked predicate violations (initialized to TRUE) are auto-corrected to FALSE. State completeness failures trigger re-enumeration with updated Ψ. Type mismatches trigger re-query of Ψ entity types. Spatial relation inconsistencies trigger a fresh perception update followed by re-grounding.

On success: P_init is locked as the L_5 artifact. The complete domain (P, A, P_init, G) is passed to the classical planner.

IV. Continuous Perception Service — Backend Options

The perception service maintains scene representation Ψ with the following structure:

```
Ψ = {
  objects: {id → ObjectRepresentation},
  surfaces: {id → SurfaceRepresentation},
  spatial_relations: [(entity₁, relation, entity₂), ...],
  last_update: Timestamp,
  confidence_scores: {element_id → confidence}
}

ObjectRepresentation = {
  geometry: {mesh, bbox, point_cloud},
  pose: SE(3),
  semantic_label: string,
  interaction_points: [{point, label, confidence, source}],
  affordance_regions: [{mask, label, confidence, source}],
  part_segments: [{mask, label, confidence}],
```

```
  articulation: {type, axis, limits} | None
}

SurfaceRepresentation = {
  geometry: {plane_eq, boundary_polygon, normal},
  semantic_label: string,
  support_region: mask,
  spatial_context: [neighboring_object_ids]
}
```

4.1 Interaction Point Detection Strategies

Ordered by descending quality (best first):

Option 1: Language-Conditioned Affordance Prediction Model *Recommended primary approach*

- **Method:** Given object crop + language query (e.g., "where to grasp this mug"), predicts affordance heatmap/regions with semantic labels
- **Output:** Affordance regions with labels and confidence scores → maps directly to Θ_sit in skill specifications
- **Strengths:**
  - Task-relevant: affordance regions are *functionally* meaningful, not just geometric features
  - Natural mapping to skill specification parameters (grasp affordance → grasp target, handle affordance → manipulation target)
  - Language conditioning enables same model to produce different regions for different actions on same object
  - Generalizes across object categories via language grounding
- **Weaknesses:**
  - Model availability — fewer mature open-source options vs. VLMs
  - Training data requirements for novel object categories
  - May need fine-tuning for specific manipulation contexts

- **Latency:** Medium (single forward pass per query, but may need multiple queries per object for different affordance types)
- **Integration:** Results populate `affordance_regions` field in ObjectRepresentation. L_4 skill specification generation queries affordance regions by label to fill Θ_sit.

Option 2: Pointing-Enabled VLM (e.g., Molmo, SoM-style prompting) *Strong alternative, best for semantic richness*

- **Method:** VLM takes object/scene image + text prompt, outputs specific points or regions with natural language part labels
- **Output:** Named points on object parts (e.g., "handle center", "lid edge", "button") with pixel coordinates → projected to 3D via depth
- **Strengths:**
  - Rich semantic labels in natural language — directly interpretable by LLM-based planners
  - Flexible: can query for arbitrary object properties and relationships
  - Zero/few-shot generalization to novel objects via language understanding
  - Can provide reasoning about *why* a point is appropriate for an action
- **Weaknesses:**
  - Higher inference latency (LLM-scale forward pass)
  - Point predictions can be spatially noisy without geometric regularization
  - Less precise than mask-based methods for region boundaries
  - Not ideal for real-time triggered updates (T2/T3) due to latency
- **Latency:** High (500ms–2s per query depending on model size)
- **Integration:** Best used during initial scene construction and on-demand sensing actions. Results populate `interaction_points` field. For time-critical updates, fall back to Option 3.
- **Hybrid strategy:** Use VLM for initial labeling pass, cache results, use faster methods for geometric refinement during execution.

Option 3: Grounded SAM / SAM2 with Semantic Labeling*Best for real-time performance and geometric precision*

- **Method:**
  - Step 1: SAM2-based segmentation produces sub-masks for object parts
  - Step 2: Lightweight classifier or small VLM labels each segment
  - For surfaces: generate masks of regions surrounding target objects using external object masks
- **Output:** Labeled part segments with precise mask boundaries → centroid/boundary points extracted for interaction points
- **Strengths:**
  - Fast inference (SAM backbone is efficient, labeling step is lightweight)
  - Precise geometric boundaries — masks give exact region extents, not just points
  - Excellent for surface detection via relational masking strategy
  - Composable: SAM segmentation can be cached and re-labeled for different queries
  - Strong for real-time background updates (T1) and triggered updates (T2/T3)
- **Weaknesses:**
  - Two-stage pipeline adds integration complexity
  - Segment labels are less semantically rich than VLM output
  - May over- or under-segment depending on prompt quality
  - Surface detection via external masking is heuristic — may miss non-obvious support relationships
- **Latency:** Low–Medium (50–200ms for segmentation, additional 50–100ms for labeling)
- **Integration:** Results populate `part_segments` field. Centroids/boundaries extracted to populate `interaction_points`. Surface masks populate

SurfaceRepresentation.

4.2 Surface Detection Strategy

Surface detection uses a relational approach independent of interaction point method:

Primary method: External context masking
- Identify target object mask (from any of the above methods)
- Generate masks of surrounding regions using depth-based plane fitting + SAM refinement
- Label support surfaces by spatial relationship to object (below → support surface, adjacent → neighboring surface)
- Populate SurfaceRepresentation with plane equations, boundaries, and object-relative labels

Key trade-off: Speed vs. accuracy
- For background updates (T1): Use depth-based plane fitting (fast, ~10ms)
- For sensing actions (T2): Use SAM-refined surface boundaries (more accurate, ~100ms)
- For failure recovery re-grounding: Full pipeline with VLM verification if needed

4.3 Recommended Hybrid Architecture

```
Background updates (1 Hz):
  SAM2 segmentation (cached) + depth-based surface detection
  → Fast, maintains geometric freshness

Initial scene construction:
  Full pipeline: SAM2 segmentation → VLM labeling → affordanc
e prediction
  → Rich semantic representation, cached in Ψ

Sensing action execution (on-demand):
  Affordance model for task-relevant regions
  OR VLM for novel/ambiguous objects
  → Targeted update, accuracy prioritized
```

```
Failure recovery re-grounding:
  Fresh SAM2 + re-query affordance/VLM as needed
  → Updated geometry with semantic verification
```

4.4 Perception Update Triggers

- **T1: Background Continuous** — Low-frequency (1 Hz) SAM2 + depth updates
- **T2: Sensing Action** — Triggered affordance/VLM query for specific predicate verification
- **T3: Precondition Failure** — Fresh perception update for failed geometric checks
- **T4: Explicit Perception** — Domain-included "perceive" actions

4.5 Latency Management

```
Planning phase:
  Uses Ψ snapshot from latest available update (cached)

Execution phase:
  Critical sensing actions → wait for fresh Ψ (affordance/VLM
query)
  Non-critical actions → use cached Ψ (SAM2 background)

Re-grounding:
  Fresh SAM2 segmentation + targeted re-labeling
```

V. Domain Knowledge Base (DKB) — Reuse Architecture

5.1 Motivation

The baseline pipeline (L1→L5) treats every task as a cold start. In practice, successive tasks in the same environment share substantial domain structure. The

DKB enables incremental extension rather than regeneration.

5.2 DKB Structure

```
DomainKnowledgeBase = {
  predicate_library: {
    predicates: {name → PredicateDefinition},
    checked_variants: {base_pred → checked_pred},
    version: int
  },

  action_schema_library: {
    schemas: {name → ActionSchema},      # Pure PDDL (L3 outp
ut)
    version: int
  },

  grounding_rule_library: {
    predicate_rules: {pred_name → grounding_rule},
    action_param_rules: {action_name → param_binding_rules},
    version: int
  },

  skill_specification_library: {
    specifications: {(action_name, object_id) → SkillSpecific
ation},
    # Each entry is a PtP JSON artifact: (π, θ_sem, θ_sit)
    # Indexed by action name + object instance
    # e.g., ("open", "bottle_1") → [move, close, twist, open,
retract]
    # Object class annotation enables cross-instance lookup
    class_index: {(action_name, object_class) → [object_id
s]},
    version: int
  },
```

```
  object_class_knowledge: {
    classes: {class_name → ObjectClassProfile},
    # Stores learned interaction strategies per object class
    # Updated when recovery discovers better approaches
  },

  exploration_strategies: {
    strategies: {(object_class, environment_region) → SkillSp
ecification},
    # Successful exploration strategies cached for reuse
    # e.g., ("dairy", "kitchen") → [go to fridge, open, obser
ve]
    # Enables future searches to start with most promising st
rategy
    version: int
  },

  execution_history: ExecutionHistoryLog,
  # Persistent log of all action attempts and outcomes
  # See §3.5.2 for schema

  environment_model: {
    surfaces: {id → SurfaceRepresentation},  # Persistent sur
faces
    static_objects: {id → ObjectRepresentation},  # Non-movea
ble objects
    robot_workspace: WorkspaceModel,
    version: int
  }
}
```

5.3 Task Execution with DKB Integration

New task pipeline:

```
execute_task_with_reuse(τ_nl, DKB, Ψ) → Result

1. L1: Goal Specification
   Generate G from τ_nl (always per-task)

2. L2: Predicate Vocabulary (with reuse)
   P_needed = predicates_required_for(G, τ_nl)
   P_existing = DKB.predicate_library.predicates
   P_missing = P_needed - P_existing

   If P_missing is empty:
     P = subset(P_existing, relevant_to(G))
     → Skip L2 generation entirely
   Else:
     P_new = generate_predicates(P_missing, G, τ_nl)
     validate(P_new)
     P = P_existing ∪ P_new
     DKB.predicate_library.add(P_new)  # Extend library

3. L3: Action Schemas (with reuse)
   A_needed = actions_required_for(G, P)
   A_existing = DKB.action_schema_library.schemas
   A_missing = A_needed - A_existing

   If A_missing is empty:
     A = subset(A_existing, relevant_to(G, P))
     → Skip L3 generation entirely
   Else:
     A_new = generate_action_schemas(A_missing, G, P, τ_nl)
     validate(A_new)
     A = A_existing ∪ A_new
     DKB.action_schema_library.add(A_new)

4. L4: Grounding + Skill Specifications (with reuse)
   For grounding rules:
```

```
      Check DKB.grounding_rule_library for existing rules
      Generate only missing rules

   For skill specifications:
      For each action α in the symbolic plan:
        target_obj = get_target_object(α, Ψ)
        key = (α.name, target_obj.id)

        # Check for exact instance match
        If key in DKB.skill_specification_library:
          Φ_cached = DKB.skill_specification_library[key]
          → Re-ground θ_sit with current Ψ (fresh poses)
          → No VLM call needed

        # Check for same object class match
        Elif (α.name, target_obj.class) in DKB.class_index:
          Φ_similar = DKB.get_best_match(α.name, target_obj.cl
ass)
          → Reuse π and θ_sem from cached specification
          → Re-select θ_sit labels if interaction points diffe
r
          → May require VLM call only for label re-selection

        # Novel — full VLM decomposition
        Else:
          Φ = vlm_decompose(α, Ψ, AMP_LIBRARY)
          DKB.skill_specification_library[key] = Φ

5. L5: Initial State Construction
   Always generated fresh from current Ψ + grounding
   (State is a snapshot, not reusable)

6. Plan, Verify, Execute (standard pipeline)
```

5.4 Reuse Levels

Component	Reuse Scope	Trigger for New Generation
Predicate vocabulary	Across all tasks in domain	Novel predicate type needed (e.g., first articulated object)
Action schemas	Across all tasks using same action types	Novel action type (e.g., first pouring task)
Grounding rules	Across all tasks in same environment	New object/surface types appear
Skill specifications	Same object instance: full reuse via re-grounding. Same object class: reuse π + Θ_sem, re-select Θ_sit. Novel class: full VLM decomposition	New object instance/class, or recovery-discovered better strategy
Environment model	Persistent (static elements)	Physical environment changes
Initial state	Never (per-task snapshot)	Every task
Goal specification	Never (per-task)	Every task

5.5 DKB Update from Recovery

When the failure recovery system discovers that a domain component is inadequate, the fix is committed back to the DKB. All action outcomes — successful or failed — are logged to the execution history regardless of recovery outcome.

```
on_action_completion(action, outcome, context):
  # Always log to execution history
  DKB.execution_history.append({
    action_name: action.name,
    target_object: action.target,
    skill_specification: action.Φ,
    outcome: outcome,
    failure_context: context if outcome == FAILURE else None,
    Ψ_before: context.Ψ_before,
    Ψ_after: context.Ψ_after,
```

```
    timestamp: now()
  })

on_recovery_success(diagnosis, repair):
  If repair.layer == L2:
    DKB.predicate_library.add(repair.new_predicates)

  If repair.layer == L3:
    DKB.action_schema_library.update(repair.modified_schemas)

  If repair.layer == L4:
    # Update skill specification for this (action, object)
    key = (repair.action.name, repair.object_id)
    DKB.skill_specification_library[key] = repair.new_specifi
cation

    # Record what went wrong for future VLM decomposition
    DKB.object_class_knowledge[repair.object_class].add_failu
re(
      failed_strategy=repair.original_specification,
      successful_strategy=repair.new_specification,
      failure_context=diagnosis
    )

  If repair.type == EXPLORATION:
    # Cache successful exploration strategy
    key = (repair.target_object_class, repair.environment_reg
ion)
    DKB.exploration_strategies[key] = repair.successful_speci
fication
```

This creates a learning loop: failures in early tasks improve skill decomposition and exploration strategies for later tasks with similar objects and environments.

VI. Hierarchical Failure Recovery Framework

6.1 Failure Taxonomy

Failure Type	Symptom	Diagnosis Target
F1: Symbolic Planning	Planner returns “no solution”	Which layer’s output is inadequate?
F2: Geometric Feasibility	Motion planning fails during compilation	Which geometric constraint fails?
F3: Execution	Primitive execution fails at runtime	Recoverable via re-grounding/replanning?
F4: Uncertainty-Induced	Sensing reveals violated assumptions	Does symbolic plan need revision?
F5: Skill Decomposition	Skill specification executes but does not achieve expected PDDL effects	Can VLM produce a better decomposition with failure context?

6.2 Failure Diagnosis Procedures

F1 — Symbolic Planning Failure Diagnosis:

```
diagnose_planning_failure(D, P_init, G) → Diagnosis

1. Check Goal Coverage → L₂ failure if predicates missing
2. Check Initial State Coverage → L₅ failure if incomplete
3. Check Action Sufficiency (backward) → L₃ failure if no act
ion produces goal predicate
4. Check Precondition Satisfaction (forward) → L₃ or L₅ gap
5. Check Sensing Coverage → L₃ failure if sensing actions mis
sing
```

F2 — Geometric Feasibility Failure Diagnosis:

```
diagnose_geometric_failure(α, Φ, constraint_failed) → Diagnos
is

1. Identify Constraint Type (reachable / collision_free / sta
ble_grasp / observable)
```

```
2. Analyze Root Cause (workspace, joint limits, obstacles, ob
ject geometry, occlusion)
3. Determine Repairability:
   - Scene-level: refresh Ψ, retry
   - Template-level: L₄ repair — change semantic params or pr
imitive sequence
   - Domain-level: L₃ repair — need different action type
   - Fundamental: L₁ — goal may be unreachable
```

F3 — Execution Failure Diagnosis:

```
diagnose_execution_failure(α, Φ, execution_error) → Diagnosis

1. Classify Error (motion error, contact error, gripper erro
r, timeout)
2. Assess Recoverability (transient → retry, parameter → re-g
round, systematic → L₄ repair)
3. Check Environment State (scene changed? belief diverged?)
4. Determine Recovery Action (re-ground, replan, repair domai
n)
```

F4 — Uncertainty-Induced Failure Diagnosis:

```
diagnose_uncertainty_failure(checked_predicate, expected, act
ual) → Diagnosis

1. Assess Plan Validity (can plan continue? must replan?)
2. Determine Replanning Scope (alternative action? goal unrea
chable?)
3. Check for Domain Inadequacy (missing earlier sensing? wron
g initial state assumptions?)
```

F5 — Skill Decomposition Failure Diagnosis:

```
diagnose_skill_failure(α, Φ, Ψ_before, Ψ_after, expected_effe
cts) → Diagnosis
```

```
1. Check Effect Achievement:
   Which expected PDDL effects were not achieved?
   Which effects were partially achieved (e.g., object moved
but not to target)?

2. Classify Failure Cause:
   - Wrong primitive sequence (mechanism was different than V
LM assumed)
   - Wrong parameters (correct sequence but wrong grasp mode,
direction, etc.)
   - Missing entity (action targeted an object not yet in Ψ —
exploration needed)
   - Environmental interference (another object blocked the a
ction)

3. Determine Re-Decomposition Viability:
   Has Ψ changed since execution? (new information available)
   Has this action been attempted before? (check execution hi
story)
   Are re-decomposition attempts within budget?
   → If YES: attempt skill re-decomposition with updated cont
ext
   → If NO: escalate to symbolic replanning or domain repair
```

6.3 Recovery Routing

```
recover_from_failure(diagnosis) → Result

Case 0: Skill Re-Decomposition (F5 — first line of defense)
  → Invoke re_decompose_skill() with failure context and exec
ution history
  → VLM generates new skill specification for same PDDL actio
n
  → If succeeds: continue plan execution
```

```
  → If exhausts re-decomposition budget: escalate to Case 1 o
r 2

Case 1: Scene-Level Recovery
  → Fresh Ψ update → re-ground → retry
  → Updates DKB: none (transient issue)

Case 2: Replanning Recovery
  → Update belief state → invoke planner from current state
  → Updates DKB: none (plan-level issue)

Case 2a: Exploration Recovery (entity missing from Ψ)
  → Generate exploration sub-problem for missing entity
  → VLM decomposes exploration action into skill specificatio
n
  → Execute → update Ψ → retry original plan step
  → If entity found: continue
  → If exploration budget exhausted: escalate to Case 3 or 4

Case 3: Layer-Specific Domain Repair
  L₁ (Goal): Unlock → relax/refine goal → regenerate L₂-L₅
  L₂ (Predicates): Unlock → add missing predicates → re-valid
ate L₃-L₅ → update DKB
  L₃ (Actions): Unlock → modify/add schemas → re-validate L₄-
L₅ → update DKB
  L₄ (Grounding/Specs): Unlock → refine rules/specs → re-vali
date L₅ → update DKB
  L₅ (Initial State): Unlock → update state construction

Case 4: Fundamental Failure
  → Report to user with diagnosis
  → Include execution history showing what was attempted

All outcomes logged to execution history.
```

```
Successful strategies cached in DKB (skill specifications + e
xploration strategies).
```

6.4 Recovery Loop with Bounded Iteration

```
execute_with_recovery(task, max_iterations=3) → Result

Iteration costs:
  Scene updates:          0     (free — cheap recovery)
  Skill re-decomposition: 0.25  (lightweight — same action, n
ew strategy)
  Exploration attempts:   0.5   (medium — physical search act
ions)
  Replanning:             0.5   (medium — new symbolic plan)
  Domain repair:          1.0   (expensive — full re-validati
on)

Hard limit prevents infinite loops.
All outcomes logged to execution history.
All successful strategies committed to DKB.
```

VII. Formal Guarantees

7.1 Layer Isolation Guarantees

- **Theorem L1 (Layer Independence):** Validated, locked layers remain unchanged during downstream repair
- **Theorem L2 (Dependency Acyclicity):** Layer dependency graph is a strict DAG: $L_1 \to L_2 \to L_3 \to L_4 \to L_5$ with Ψ feeding L_4
- **Theorem L3 (Validation Monotonicity):** Locked layers remain valid until explicitly unlocked

7.2 Recovery Guarantees

- **Theorem R1 (Failure Localization):** Every failure attributable to specific layer, skill decomposition level, or external factor. Extended to five failure classes: symbolic (F1), geometric (F2), execution (F3), uncertainty-induced (F4), and skill decomposition (F5).
- **Theorem R2 (Minimal Repair Scope):** Recovery unlocks only minimum necessary layers. Skill re-decomposition (F5) does not unlock any layer — it operates entirely within L_4's execution context.
- **Theorem R3 (Recovery Termination):** Bounded iteration with cost-weighted counting. Scene updates: 0 cost (unlimited). Skill re-decomposition: 0.25 per attempt. Exploration: 0.5 per attempt. Replanning: 0.5. Domain repair: 1.0. For max_iterations=k, total re-decomposition attempts ≤ 4k, exploration attempts ≤ 2k, replanning ≤ 2k, domain repairs ≤ k.

7.3 Perception-Domain Decoupling Guarantees

- **Theorem P1 (Grounding Rule Stability):** Rules reference scene element types, remain valid across Ψ updates
- **Theorem P2 (Asynchronous Perception Safety):** Pre-execution verification detects Ψ changes, blocks unsafe execution

7.4 Reuse Guarantees

- **Theorem D1 (Monotonic Domain Extension):** Adding new predicates/actions for a new task does not invalidate existing validated artifacts. *Proof sketch:* New additions extend sets P and A without modifying existing definitions. Existing validation (symbolic closure, parameter consistency) is preserved because existing schemas reference only pre-existing predicates, and new schemas are validated independently.
- **Theorem D2 (Skill Specification Reuse Correctness):** A skill specification Φ = (π, Θ_sem, Θ_sit) generated for action α on object o_1 in scene Ψ_1 can be reused for the same action on object o_2 of the same class in scene Ψ_2 by re-grounding Θ_sit. *Proof sketch:* The structural specification (π, Θ_sem) depends on the action intent and object mechanism/geometry, which are shared within an object class. Only the situational parameters Θ_sit require re-resolution with current Ψ, which is performed during L_4 re-grounding.

- **Theorem D3 (Recovery-Driven Improvement):** If recovery discovers a better skill specification for (action, object), all future tasks involving the same object benefit. *Proof sketch:* DKB update replaces the skill specification entry. Future L_4 lookups retrieve the improved specification. Object class knowledge records failure context, preventing regression.

7.5 Compositional Safety

- **Theorem S1 (End-to-End Safety):** All layer validations compose to system-wide safety guarantees
- **Theorem S2 (Execution Safety with Recovery):** Preconditions verified, constraints checked, recovery bounded

7.6 What We Do NOT Guarantee

- **N1:** LLM semantic correctness
- **N2:** Optimal plans
- **N3:** Perception accuracy
- **N4:** Task completability
- **N5:** Recovery success
- **N6:** DKB consistency across major environment changes (DKB may need partial invalidation if environment changes significantly)
- **N7:** Exploration success (the target object may not exist in the environment; bounded exploration attempts may not cover the right locations)
- **N8:** Skill decomposition correctness (the VLM may produce a primitive sequence that is structurally valid but semantically wrong for the task; this is caught at execution time via effect verification, not at validation time)

VIII. Summary of Key Contributions

C1: Five-Layer Domain Generation with Clean Symbolic/Execution Separation
L_3 produces pure PDDL action schemas; L_4 uses VLM-based primitive decomposition (following the Prompt-to-Primitives approach) to generate scene-grounded skill specifications. This enables independent evolution of symbolic

planning and execution strategies, with full flexibility to adapt to novel objects and mechanisms.

C2: Perception Service with Configurable Backends
Three ranked perception strategies (affordance models, pointing VLMs, SAM-based segmentation) with a hybrid architecture balancing semantic richness and real-time performance.

C3: Domain Knowledge Base for Cross-Task Reuse
Monotonically extending knowledge base persists predicates, schemas, grounding rules, skill specifications, and exploration strategies across tasks. New tasks query existing knowledge first, generating only missing components.

C4: Skill Re-Decomposition and Active Exploration
When a skill specification fails to achieve expected effects, the system re-decomposes at the L_4 level before escalating to heavier recovery — generating a new primitive sequence informed by the execution history of what was previously attempted and what failed. For missing entities, the VLM composes exploratory skill specifications using observation and navigation primitives, with successful strategies cached for future reuse.

C5: Execution History as Persistent Context
All action attempts and outcomes are recorded in a persistent log that is fed to the VLM during skill decomposition and re-decomposition, preventing repeated failures and enabling systematic exploration. The execution history bridges individual action attempts into a coherent learning signal across tasks.

C6: Hierarchical Failure Recovery Framework
Five-class failure taxonomy (symbolic, geometric, execution, uncertainty-induced, skill decomposition) with diagnostic procedures routing failures to appropriate abstraction layers for minimal-scope repair with bounded iteration.

C7: Compositional Safety Guarantees
Layer-specific guarantees compose to system-wide safety, extended with reuse correctness (monotonic extension, skill specification reuse, recovery improvement) theorems.

IX. Open Questions and Next Steps

1. **DKB invalidation policy:** When does accumulated knowledge become stale? Need criteria for partial DKB reset when environment changes significantly.
2. **Object class taxonomy:** How granular should object classes be for skill specification reuse? “mug” vs. “container_with_handle” vs. “graspable_object” — granularity affects reuse rate vs. specification quality.
3. **Affordance model selection:** Specific model choice for Option 1 perception backend. Evaluate available language-conditioned affordance models for manipulation-relevant categories.
4. **DKB query mechanism:** How does the system determine which existing predicates/schemas are “relevant” to a new task? This is itself an LLM reasoning step that needs specification.
5. **Cross-environment transfer:** Can DKB knowledge (especially action schemas and skill specifications) transfer to new environments, or is it environment-specific?
6. **Formal evaluation plan:** Metrics for measuring reuse effectiveness (cold-start time vs. warm-start time, recovery rate improvement over successive tasks, DKB growth curves).
7. **Exploration strategy quality:** How do we measure whether the VLM’s exploration strategies are efficient? A VLM that checks every cabinet sequentially is correct but inefficient compared to one that reasons about likely object locations. Metrics could include number of exploration steps before entity found, or comparison against an oracle that knows the environment layout.
8. **Execution history management:** The execution history grows unboundedly across tasks. Need a policy for summarization or pruning — which entries are relevant for a given VLM prompt? Passing the full history would exceed context limits. Options include recency-weighted filtering, failure-only filtering, or LLM-based summarization of history into compact strategy notes.
9. **Contingent planner integration:** The sensing coverage invariant produces domains suited for contingent planning (branching on observation outcomes). Specific planner choice and integration strategy — e.g., FOND planners, online

replanning, or conditional plan compilation — needs evaluation for manipulation-scale domains.